

THREAT INTELLIGENCE FOR SOC

Contents

Threat Intelligence for SOC	2
What is Threat Intelligence?	2
Consumers and Producers	3
Producers	3
Consumers	3
Consuming Threat Intelligence	4
Intelligence-driven Prevention	7
IP blocking via firewall.....	8
Domain blocking through email gateways	8
Domain blocking through DNS sinkhole	9
Intelligence-driven Detection.....	12
Optimizing detection capabilities	12

Threat Intelligence for SOC

As a group working to secure an organization, the security team is expected to be prepared to handle the never-ending evolution of threats and anticipate unknown possibilities of potential compromises.

We need to ask these questions:

- Is our organization prepared to handle emerging threats like new malware IOCs or zero days?
- Can we determine unknown adversaries or apply known indicators from reliable sources in our SOC pipeline?

What is Threat Intelligence?

Threat Intelligence is the analysis of data and information using tools and techniques to generate meaningful patterns to mitigate against potential risks associated with existing or emerging threats targeting organizations, industries, sectors or governments.

4 classifications of TI:

- 1) Strategic Intel: High-level intel that looks into the organization's threat landscape and maps out the risk areas based on trends, patterns and emerging threats that may impact business decisions.
- 2) Technical Intel: Examines evidence and artefacts of attacks an adversary uses. Incident Response teams can use this intel to create a baseline attack surface to analyze and develop defense mechanisms.
- 3) Tactical Intel: Assesses adversaries' tactics, techniques and procedures (TTPs). This intel can strengthen security controls and address vulnerabilities through real-time investigations.
- 4) Operational Intel: Assesses an adversary's specific motives and intent to perform an attack. Security teams may use this intel to understand the critical assets available in the organization (people, processes and technologies) that threat actors may target.

We will focus on the most common of the four, which is Technical Intel, for utilizing artefacts generated by adversaries to improve the Security Operations pipeline. This type is also mainly known as IOC-based Threat Intelligence.

Consumers and Producers

= do we build the knowledge base, or do we consume the knowledge of others?

The common notion of Threat Intelligence is the dataset of known bad IOCs collated by different entities. It may be malicious URLs hosting malware or IP addresses of suspicious connections. But would you know how this information is gathered for the disposal of security analysts?

Producers

- people who gather, analyze and disseminate threat intelligence data for others and themselves.
- they create reports, advisories, and resources that are shared within the broader cybersecurity community
- This group includes cybersecurity vendors, research labs and organizations specializing in collecting and interpreting data on emerging cyber threats

How do they get the data for Threat Intel?

Simply by monitoring network, which involves monitoring an organization's network traffic to identify abnormal behaviour from the inside or a honeypot server exposed externally

The results of these collections are then further analyzed, attributed to potential threat actors and published eventually to help other organizations.

Consumers

On the other hand, Threat Intelligence Consumers are organizations or individuals who consume Threat Intelligence created by Producers. The information gathered from different source is utilized to improve the organization's security posture.

How does this group typically leverage the intelligence data shared with them?

- Identifying vulnerabilities (for example: CVE publications)
- Prevention and Detection (for example: Using IOCs to prevent intrusions by blocking these artefacts or detect them by applying them to threat detection rules)
- Incident Response (for example: To respond more effectively thanks to intelligence data)
- Collaborating with others (for example: Sharing the newly discovered intelligence to security analysts etc.)

Consuming Threat Intelligence

For the conclusion, we are going to hunt and consume IOCs provided by Threat Intelligence Producers.

For this task we will also use web software called Uncoder.io, which helps us transform Sigma rules or IOC lists into usable rules.

For IOCs, the tool accepts six different types of IOCs, namely:

Ips, Domains, URLs, Hashes, Emails, Files

We use this set of Ips and feed them into Uncoder:

117[.]213[.]7[.]8

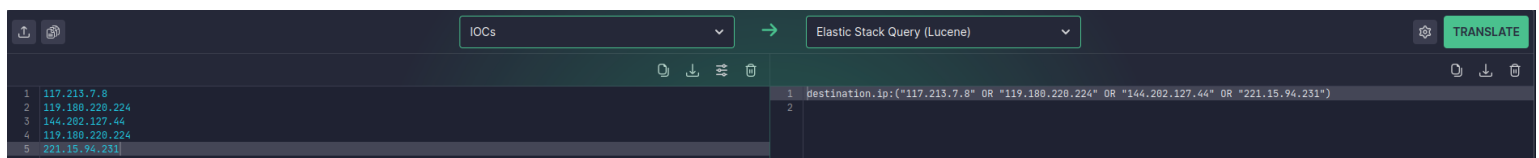
119[.]180[.]220[.]224

144[.]202[.]127[.]44

119[.]180[.]220[.]224

221[.]15[.]94[.]231

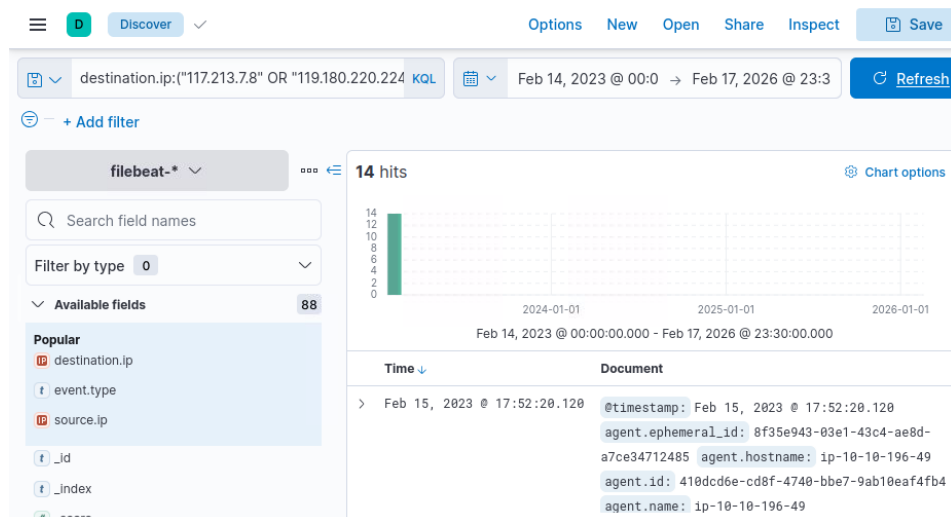
Then we have selected IOCs and to transform them into the target platform which is Elastic Query since the provided SIEM is built on that and we translate them.



Then we copy the result which is:

destination.ip:("117.213.7.8" OR "119.180.220.224" OR "144.202.127.44" OR "221.15.94.231")

We will paste it into the Kibana query in Discover feature. For our purpose the query must be under filebeat-* index and searches between 14.2.2023 – 17.2.2023.



And now for the real challenge we have acquired list of our own IPs:

135[.]181[.]103[.]89

185[.]224[.]126[.]215

185[.]224[.]128[.]215

171[.]24[.]136[.]15

171[.]22[.]136[.]15

195[.]133[.]40[.]108

103[.]190[.]37[.]169

103[.]170[.]37[.]169

103[.]190[.]37[.]169

185[.]224[.]128[.]215

107[.]175[.]202[.]151

107[.]175[.]202[.]158

195[.]133[.]40[.]108

107[.]175[.]202[.]158

109[.]206[.]240[.]194

1) How many unique IP addresses were provided in the IOC list?

The Uncoder tool is great in the aspect of counting only the unique IPs. (11)

	IOCs
1	135.181.103.89
2	185.224.126.215
3	185.224.128.215
4	171.22.136.15
5	171.22.136.15
6	195.133.40.108
7	103.190.37.169
8	103.170.37.169
9	103.190.37.169
10	185.224.128.215
11	107.175.202.151
12	107.175.202.158
13	195.133.40.108
14	107.175.202.158
15	109.206.240.194
16	

0 Hashes 0 Domains 0 URLs 11 IPs 11/10000

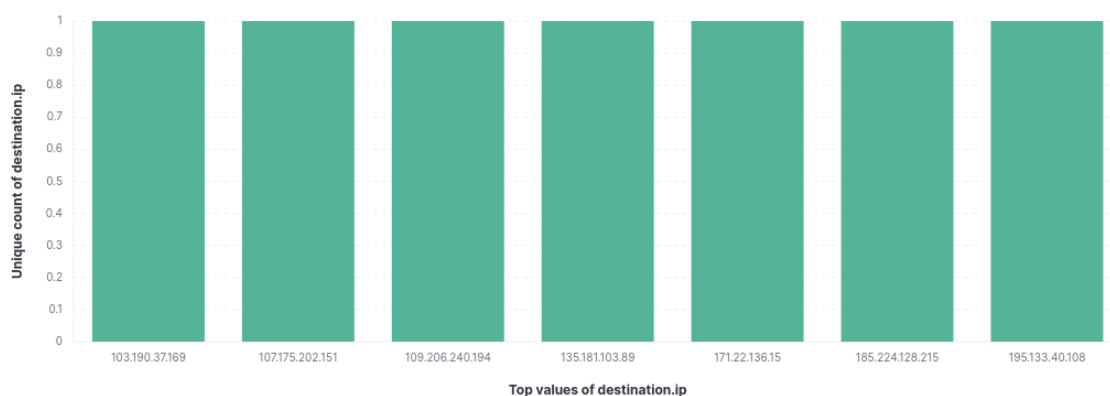
2) Based on the set of IOCs, how many IOC hits were discovered in the logs?

destination.ip:("135.181.103.89" OR "185.224.126.215" OR "185.224.128.215")

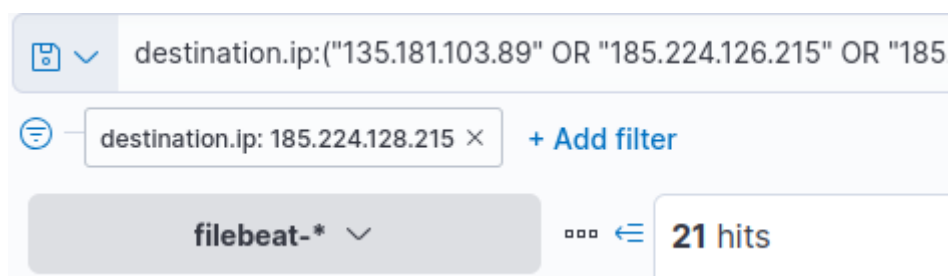
+ Add filter

filebeat-* 48 hits

3) Out of the total number of IOCs, how many unique IP addresses were discovered in the logs? 7 IP addresses



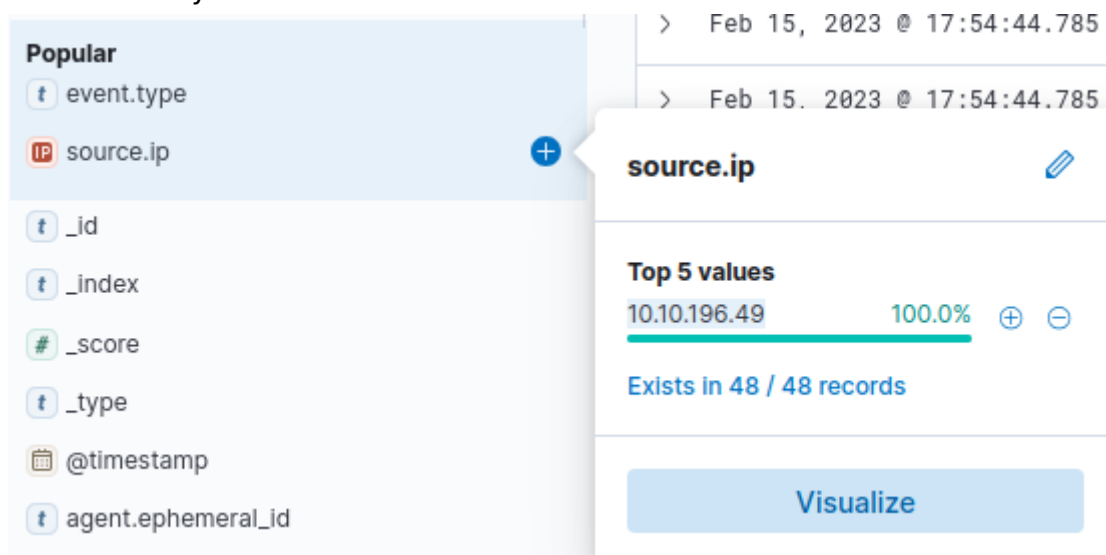
4) How many connections were made to 185[.]224[.]128[.]215?



We have chosen our destination.ip as 185.224.128.215.

5) What is the IP address of the compromised host?

There is only one source.ip so it must be it, because it occurs to me that this was C2 attack, and the source ip is our workstation and the destination ip, at least some of them are most likely the attack server or other malicious domains:



6) What is the destination port of connections made to 107[.]175[.]202[.]151?

> Feb 15, 2023 @ 17:54:22.016	107.175.202.151	80
> Feb 15, 2023 @ 17:54:22.016	107.175.202.151	80
> Feb 15, 2023 @ 17:54:21.927	107.175.202.151	80

It's port 80.

Intelligence-driven Prevention

Using our current knowledge of Threat Intelligence, we will utilise the IOCs from reliable sources to deploy security controls that will prevent malicious activity in our infrastructure.

Domains: Typically attributed to URLs used to host malicious files, C2 callbacks or email domains used for spam.

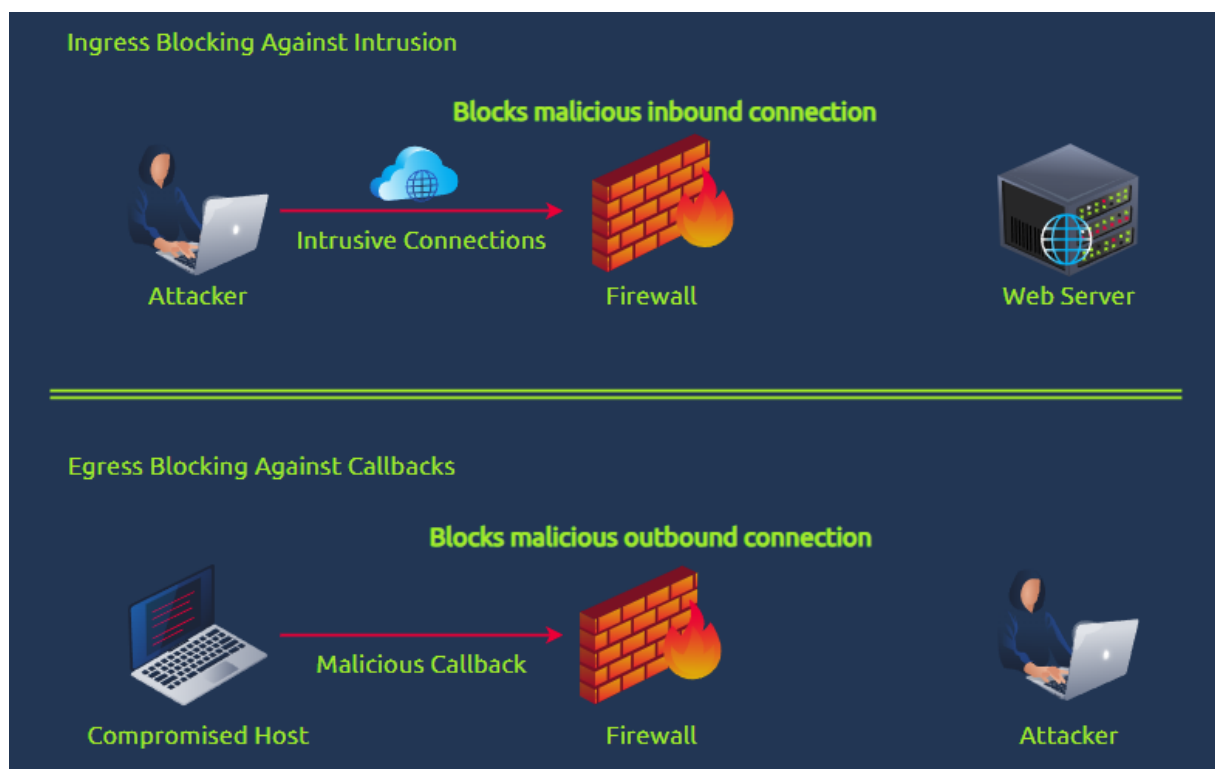
IP addresses: Commonly attributed to addresses known to execute attacks seen from external assets or outbound callbacks from malware.

IP blocking via firewall

IP blocking is a well-known security measure that involves blocking ingress or egressing network traffic based on the device's IP address attempting to initiate a network connection. It is typically done using a Firewall, a security system that controls the traffic based on predetermined rules.

Configuring firewall rules could be overwhelming, but having a direction to deny connections from a known malicious IP address is a good start in preventing malicious connections, such as:

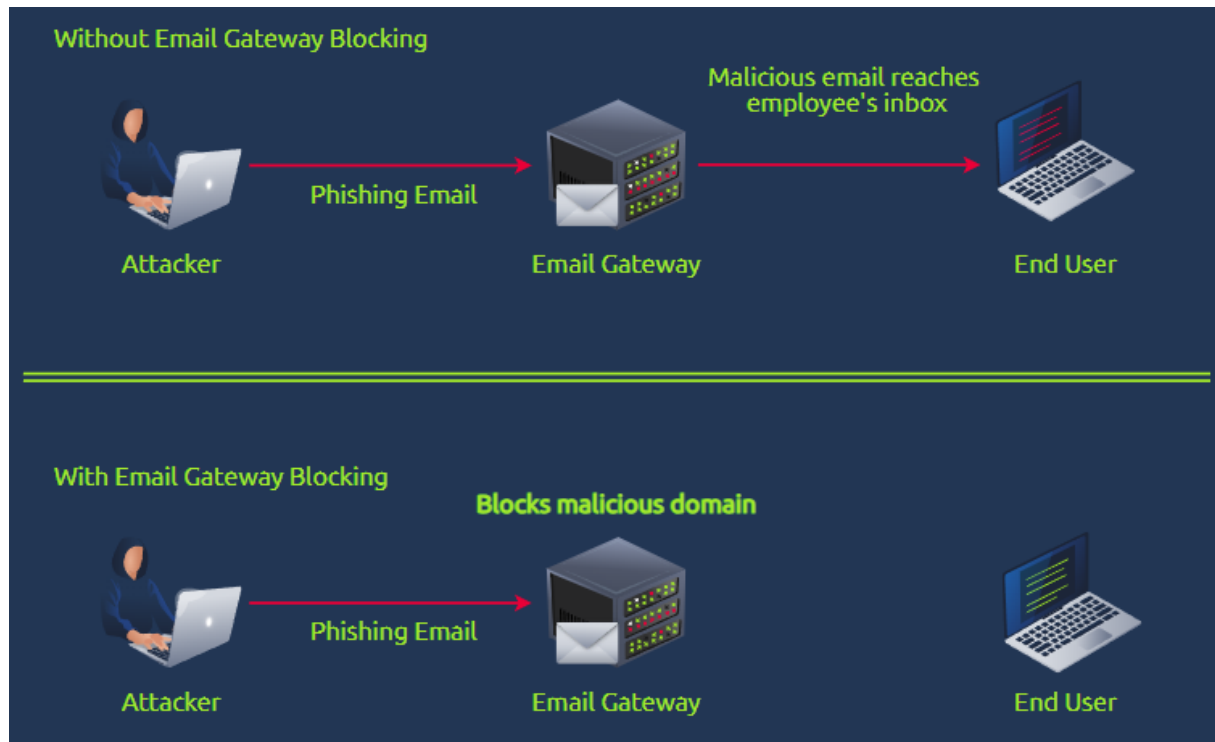
- prevent intrusive connections against external applications that may affect service uptime or compromise via a known vulnerability
- prevent connection attempts to the threat actor's infrastructure after successful malware execution



Domain blocking through email gateways

Similar to IP blocking, we can configure Email Gateways to prevent known malicious domains from forwarding incoming emails messages based on the sender's domain.

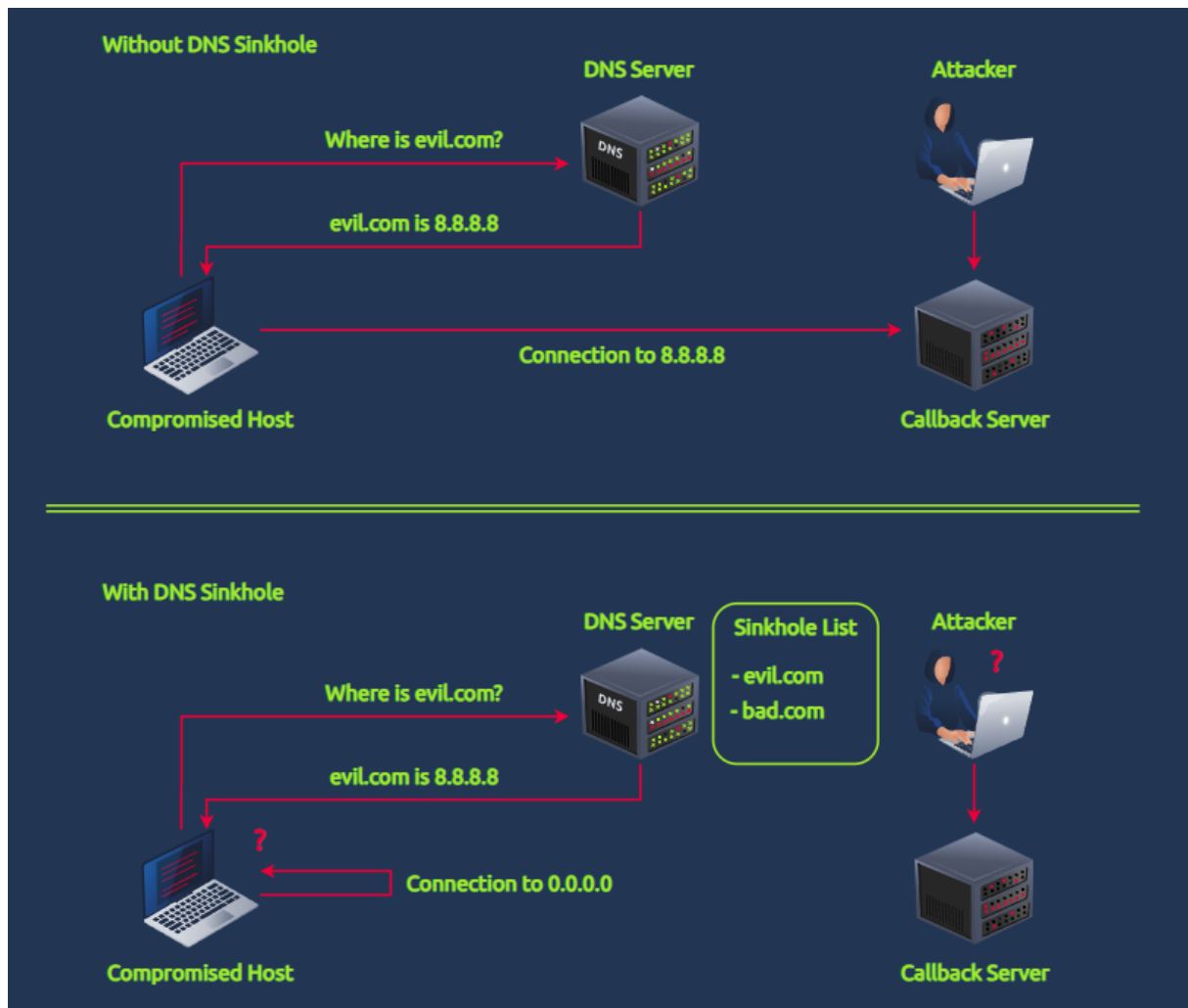
Email gateways also depend on a ruleset, which should contain the block list of domains known to send spam or phishing emails. Once the block list is populated, the email gateway prevents threat actors from reaching the inbox of the target users in the organization.



Preventing spam emails from reaching employees' inboxes reduces the potential attack surfaces of a threat actor in compromising the organization. Most of the time, an organization takeover starts with the execution of a malicious attachment or submitting credentials to a phishing website. An additional prevention layer slightly reduces the burden on the users' phishing awareness capabilities.

Domain blocking through DNS sinkhole

DNS Sinkhole is a security measure that mitigates connections to a malicious domain. This is typically done by redirecting all DNS requests from a known malicious domain to a sinkhole, preventing the resolution to their counterpart IP addresses.

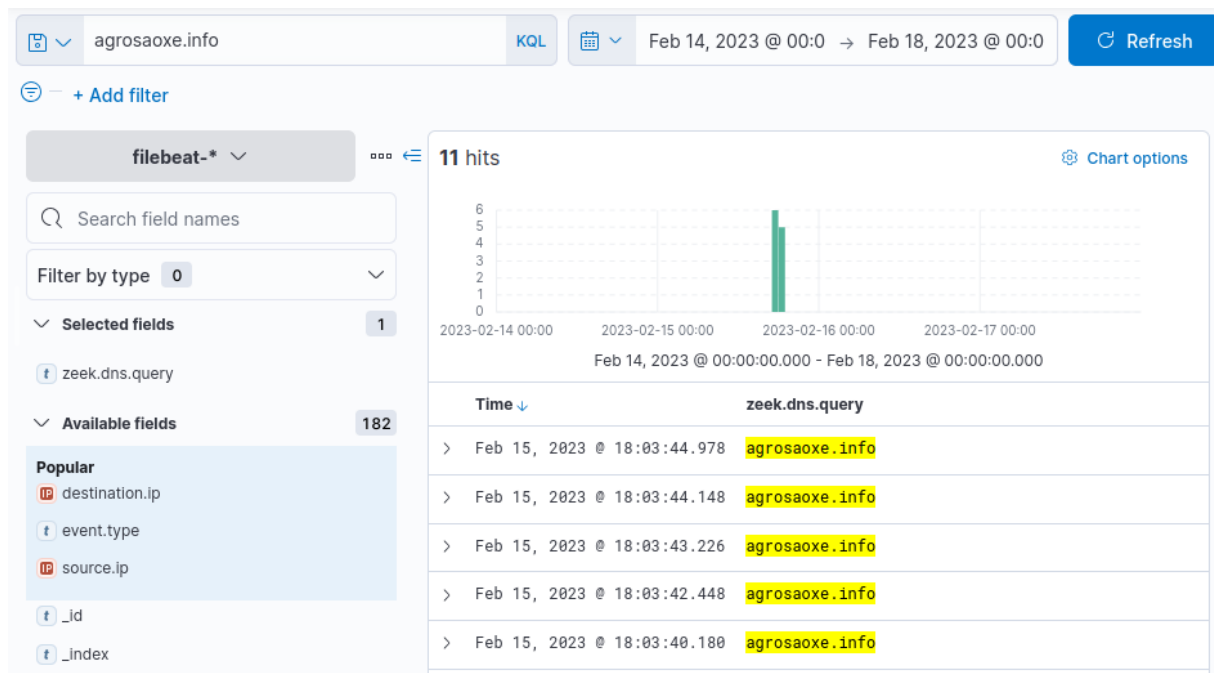


Using the logs generated by the SIEM, you are tasked to hunt all domains identified as malicious by your DNS Sinkhole. The Threat Intelligence team has provided a single domain to start with your activity: agrosaoxe[.]info (we must remove the defang)

In addition, we may also need the following KQL query templates to complete the investigation:

- dns.question.name: "replace with domain"
- dns.answers.data: "replace with sinkhole IP"

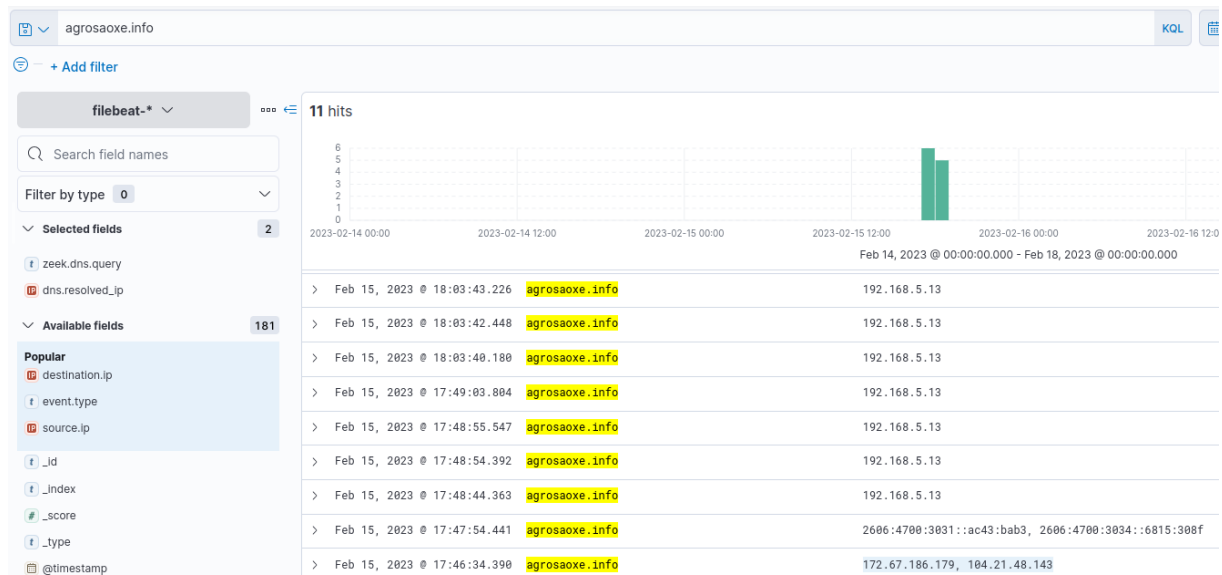
1) How many DNS queries to agrosaoxe[.]info have been created?



Into the query we have put agrosaoxe.info and select field dns.query.

- Before deploying the sinkhole configuration, what IPv4 addresses are resolved by agrosaoxe[.]info? (format: IPs in ascending order)

So at the very beginning the agrosaoxe.info resolved into 172.67.186.179, 104.21.48.143.



- What is the IP address used for DNS Sinkhole?

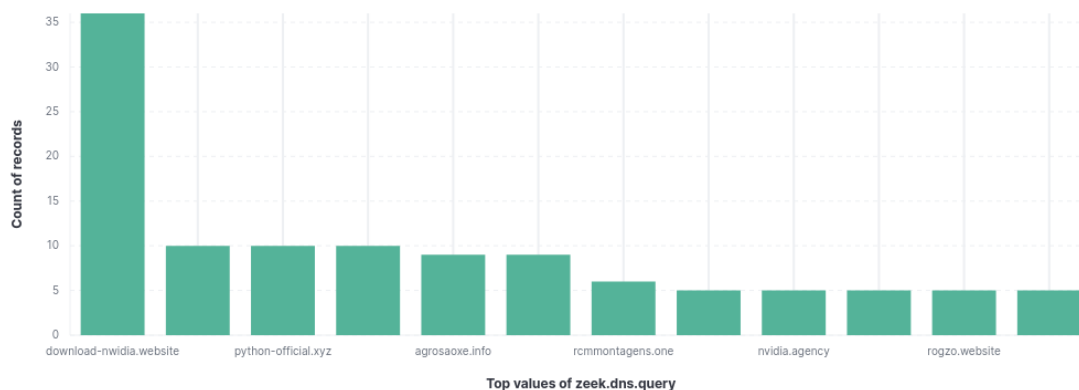
192.168.5.13, in the logs above we can see the changed IP address after we have applied the sinkhole IP address.

4) How many hits were caused by connections to sinkholed domains?

We put the sinkhole IP address into our search query and gotten 115 hits.

5) How many unique domains have been sinkholed?

We used the zeek dns query visualization method. (12)



Intelligence-driven Detection

In this task, we must improve the detection capabilities of our tooling.

Optimizing detection capabilities

Implementing detection based on IOCs may be straightforward, as one may think we can deploy a blocklist rule for known malicious IOCs. An example set of detection use cases is listed below for each common Threat Intel IOC.

IP address	Connections via firewall logs wherein the direction of the connection dictates the potential root cause: - Egress connection to a malicious IP indicated a potential execution of malware, thus communicating with a threat actor's IP address (post-exploitation phase) - Ingress connection from a malicious IP dictates an intrusion attempt from a malicious actors, showing traces of the pre-exploitation phase. (brute force attack)
URL	Connections via proxy logs wherein the HTTP method dictates the nature of the connection: - HTTP GET requests indicate a potential download of malicious files or access to a phishing website

	- Moreover, HTTP POST request indicate a potential submission of credentials or exfiltration of stolen files
Domain	Malicious domain seen in DNS logs directly indicate a malicious activity in either of the following: - The domain hosts malware or additional files for its execution chain - The domain is a phishing website - The domain is being used for a C2 connection

The scenarios above depict the usage of publicly available IOCs to hit any suspicious connections across different data sources such as firewalls, DNS and proxy server. However, this kind of setup may require continuous fine-tuning of rules to accommodate the growth of IOCs.

Prevention techniques would be:

- DNS Sinkhole
- Firewall IP blocking
- Proxy BLOCKING
- Mail gateway blocking

Let's go back to the Sigma, which is an open-source generic signature language to describe log events in a structured format. This allows for quick sharing of detection methods by security analysts.

We will use this Sigma rule to hunt for sinkholed domains:

```

title: DNS Sinkhole
author: TryHackMe User
description: Sigma rule for sinkholed DNS queries
logsource:
  category: dns
detection:
  select_sinkholed:
    dns.resolved_ip:
      - '0.0.0.0'
  condition: select_sinkholed
falsepositives:
  - Unknown
status: experimental
level: medium
tags:
  - dns
  - filebeat

```

It hunts for all DNS queries resolved into 0.0.0.0. So this indicates possible connection to malicious websites therefore thanks to the sinkhole applied ended up as a failure.

We will use this rule to translate it into ElastAlert.

```
alert:
- debug
description: Sigma rule for sinkholed DNS queries. Author: TryHackMe User. Rule ID: a726c44d-a1f6-4d93-a8da-10b491de72a2.
  License: DRL 1.1.
filter:
- query_string:
  query: index="logs-*" AND dns.resolved_ip:0\.\0\.\0\.\0
index: winlogbeat-*
name: DNS Sinkhole
priority: 3
realert:
  minutes: 0
type: any
```

What is ElastAlert?

It's an open-source framework for alerting on anomalies, spikes or other patterns of interest found in data stored in Elasticsearch. It integrates with Elasticsearch, Kibana and other tools in the Elasticsearch ecosystem and can be configured to send alerts to various external services such as email, slack, pagerduty...

The alert consists of:

alert	The alerter type of use. The value debug will log the alert information at the info level
filter	A list of Elasticsearch query filters. The current query searches for all domains resolving to 0.0.0.0
index	The name of the index that will be searched. In our current context (filebeat-*)
realert	This option allows you to ignore repeating alerts for some times. The value is minutes: 0, because it will generate all alerts even redundant ones.
type	The rule type to use. The value any will generate an alert for every successful query return.

Now we will navigate to our machine via SSH. and to the /elastalert directory.

```
user@ip-10-113-140-169:~$ cd elastalert/
user@ip-10-113-140-169:~/elastalert$ ls
config.yaml  rules
user@ip-10-113-140-169:~/elastalert$
```

We can see that configuration file and set of rules are waiting for us.

The config.yaml file contains all the configurations needed to connect and query to our Elasticsearch instance, while the rules directory contains a placeholder rule.

We will now add our rule from Uncoder.io to the rules directory.

```
File Edit View Search Terminal Help
GNU nano 4.8 sinkhole.yaml
alert:
- debug
description: Placeholder
filter:
- query_string:
    query: zeek.dns.answers:"placeholder"
index: filebeat-*
name: Placeholder
priority: 2
realert:
  minutes: 0
type: any
```

This is just an example which we can overwrite by our rule.

This is the corrected version:

```
user@ip-10-113-140-169: ~/elastalert/rules
File Edit View Search Terminal Help
GNU nano 4.8 sinkhole.yaml Modified
alert:
- debug
description: Sigma rule for sinkholed DNS queries.
filter:
- query_string:
    query: dns.resolved_ip:"0.0.0.0"
index: filebeat-*
name: DNS Sinkhole
priority: 3
realert:
  minutes: 0
type: any
```

And now we will start executing ElastAlert.

This is the command for ElastAlert:

elastalert --start 2023-02-16T00:00:00 --verbose 2>&1 | tee output.txt

It executes the rule we configured starting from 16.2.2023 until now. And the |tee output.txt is to write the results into output.txt. It takes us a couple of minutes, because it runs the rule until today – 20.6.2026.

```
}
tags: [
  "zeek.dns",
  "_geoip_expired_database"
]
zeek: {
  "dns": {
    "AA": false,
    "RA": false,
    "RD": false,
    "TC": false,
    "TTLs": [
      3600
    ],
    "answers": [
      "0.0.0.0"
    ],
    "query": "myanmarfuturescience.com",
    "rcode": 0,
    "rcode_name": "NOERROR",
    "rejected": false,
    "trans_id": "62001"
  },
  "session_id": "C2yPzG4sF038022N58"
}

INFO:elastalert:Ran dns_sinkhole from 2023-02-16 00:00 UTC to 2026-06-20 15:43 UTC: 0 query hits (0 already seen), 40 matches, 40 alerts sent
WARNING:root:Querying from 2023-02-16 00:00 UTC to 2026-06-20 15:43 UTC took longer than 0:01:00!
```

So there were 40 alerts sent in total and the number of unique caught malicious domains that ended up being sinkholed are:

```
user@ip-10-113-140-169:~/elastalert$ grep '"query":' output.txt | sort | uniq
"query": "ball-kit.click",
"query": "bar.com",
"query": "foo.bar.com",
"query": "myanmarfuturescience.com",
"query": "twizt.ru",
"query": "uradlimited.com",
"query": "www.ingetic.cl",
```

Because the domains were saved in query and we wanted to know only the unique ones and we sorted them alphabetically, otherwise the command won't work properly.

(the reason: uniq command eradicates duplicates only when they go one after another, which means the alphabetical order is important for the command to work properly and see that there are more domains under same name.)

Example:

```
user@ip-10-113-140-169:~/elastalert$ grep '"query":' output.txt | sort | uniq
"query": "ball-kit.click",
"query": "bar.com",
"query": "foo.bar.com",
"query": "myanmarfuturescience.com",
"query": "twizt.ru",
"query": "uradlimited.com",
"query": "www.ingetic.cl",
user@ip-10-113-140-169:~/elastalert$ grep '"query":' output.txt | uniq
"query": "foo.bar.com",
"query": "bar.com",
"query": "ball-kit.click",
"query": "uradlimited.com",
"query": "twizt.ru",
"query": "www.ingetic.cl",
"query": "myanmarfuturescience.com",
"query": "ball-kit.click",
"query": "uradlimited.com",
"query": "twizt.ru",
"query": "www.ingetic.cl",
"query": "myanmarfuturescience.com",
"query": "ball-kit.click",
"query": "uradlimited.com",
"query": "twizt.ru",
"query": "www.ingetic.cl",
"query": "myanmarfuturescience.com",
"query": "ball-kit.click",
"query": "uradlimited.com",
"query": "twizt.ru",
"query": "www.ingetic.cl",
"query": "myanmarfuturescience.com",
"query": "ball-kit.click",
"query": "uradlimited.com",
"query": "twizt.ru",
"query": "www.ingetic.cl",
```